INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCIÓN AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	PERSISTENCIA DE DATOS CON ROOM DATABASE EN JETPACK COMPOSE				
NÚMERO DE PRÁCTICA:	9	**AÑO LECTIVO:**	2026-B	**NRO. SEMESTRE:**	VIII
FECHA DE PRESENTACIÓN	31/10/2026	**HORA DE PRESENTACIÓN**	23:59		
INTEGRANTE (s): - Zuñiga Villacorta, Peter Sebastian				**NOTA (0-20)**	
DOCENTE(s): Roxana Evelyn Limache Calatayud					

RESULTADOS Y PRUEBAS

Github Repo: https://github.com/UNSA-IS-2026-B/idnp_lab/tree/main/lab09

I. OBJETIVOS

- Comprender los componentes de Room (Entity, DAO, Database) y su relación con SQLite.
- Diseñar un modelo de datos con entidades relacionadas para persistir información de los edificios del campus.
- Implementar operaciones CRUD y una consulta relacional entre entidades mediante Room.

II. EJERCICIOS RESUELTOS

A continuación, se presenta la solución paso a paso estructurada técnicamente, detallando las decisiones de arquitectura, configuración y funcionamiento interno del motor de persistencia.

Paso 1 y 2: Configuración del Entorno y Gradle con KSP (Kotlin Symbol Processing)

Para trabajar con Room en proyectos Android modernos se utiliza KSP en lugar de KAPT. KSP analiza el árbol de sintaxis de Kotlin directamente sin generar código intermedio de Java, optimizando los tiempos de compilación.

1. Archivo gradle/libs.versions.toml

Se declaran los plugins y librerías centralizados garantizando la compatibilidad entre el plugin de Android (AGP), Kotlin y KSP:

```
Kotlin
[versions]
agp = "9.3.3"
kotlin = "2.2.10"
ksp = "2.3.6"

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

2 Archivo build.gradle.kts (Proyecto raíz)

Aplica los plugins sin instanciarlos de inmediato (apply false):

```
Kotlin
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.compose) apply false
    alias(libs.plugins.ksp) apply false
}
```

3. Archivo build.gradle.kts (Módulo :app)

Se activan los plugins y se agregan las dependencias runtime y de compilación:

```
Kotlin
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)
}

dependencies {
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.11.0")

    // Room Database
    implementation("androidx.room:room-runtime:2.7.2")
    implementation("androidx.room:room-ktx:2.7.2")
    ksp("androidx.room:room-compiler:2.7.2")
}
```

- room-runtime: Contiene el motor base, la API abstracta y las anotaciones.
- room-ktx: Aporta extensiones de Corrutinas para soportar funciones suspendidas (suspend) e integración con Flow.

- room-compiler (vía ksp): Genera automáticamente el código en Kotlin/Java que implementa las interfaces @Dao y @Database durante el proceso de compilación (Rebuild Project).

Paso 3, 4 y 5: Modelado de Entidades Relacionadas (Entity)

Las clases annotated con @Entity representan las tablas físicas dentro del archivo de base de datos SQLite.

1. Entidad Padre: EdificioEntity

```kotlin
@Entity
data class EdificioEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nombre: String,
    val categoria: String
)
```

2. Entidad Hija: AmbienteEntity (Con Clave Foránea)

```kotlin
@Entity(
    foreignKeys = [
        ForeignKey(
            entity = EdificioEntity::class,
            parentColumns = ["id"],
            childColumns = ["edificioId"]
        )
    ]
)
data class AmbienteEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val edificioId: Int,
    val nombre: String
)
```

- Clave Primaria (@PrimaryKey(autoGenerate = true)): Permite que el motor SQLite gestione la secuencia de IDs de manera automática.
- Clave Foránea (@ForeignKey): Vincula la columna edificioId de la tabla AmbienteEntity con la columna id de EdificioEntity. Esto impone una restricción de integridad referencial directamente en la base de datos: no es posible registrar un ambiente con un edificioId inexistente**.**

Paso 6 y 7: Capa de Acceso a Datos (DAO) y Consultas Relacionales

El DAO define la interfaz abstracta mediante la cual el código de la aplicación realiza operaciones contra la base de datos sin escribir SQL manual imperativo.

1. Data Transfer Object (DTO): EdificioConAmbientes

Room exige definir un contenedor intermedio para estructurar respuestas que combinan dos entidades en una relación 1:N.

Kotlin

```kotlin
data class EdificioConAmbientes(
    @Embedded val edificio: EdificioEntity,
    @Relation(
        parentColumn = "id",
        entityColumn = "edificioId"
    )
    val ambientes: List<AmbienteEntity>
)
```

2. Interfaz DAO Completa: EdificioDao

Kotlin

```kotlin
@Dao
interface EdificioDao {
    @Insert
    suspend fun insertarEdificio(edificio: EdificioEntity): Long

    @Insert
    suspend fun insertarAmbiente(ambiente: AmbienteEntity)

    @Update
    suspend fun actualizarEdificio(edificio: EdificioEntity)

    @Delete
    suspend fun eliminarEdificio(edificio: EdificioEntity)

    @Query("SELECT * FROM EdificioEntity")
    suspend fun obtenerTodos(): List<EdificioEntity>

    @Transaction
    @Query("SELECT * FROM EdificioEntity")
    suspend fun obtenerConAmbientes(): List<EdificioConAmbientes>
}
```

- suspend Functions: Indican que la ejecución de la consulta se traslada a un hilo de I/O en segundo plano mediante Corrutinas, evitando congelar la interfaz de usuario.
- @Embedded: Desempaqueta todos los campos de EdificioEntity dentro de la estructura combinada.
- @Relation: Informa a Room cómo vincular el campo id del padre con el campo edificioId de los hijos.
- @Transaction: Fundamental para consistencia de datos. Debido a que @Relation realiza dos consultas SQLite separadas (una para edificios y otra para ambientes), @Transaction asegura que ambas se

ejecuten en un bloque atómico único. Esto evita lecturas inconsistentes (race conditions) si la base de datos es modificada mientras se realiza la consulta.

Paso 8: Configuración de la Base de Datos (RoomDatabase)

Se crea la clase abstracta de la base de datos y la función de inicialización mediante databaseBuilder.

Kotlin

```
@Database(entities = [EdificioEntity::class, AmbienteEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun edificioDao(): EdificioDao
}

fun crearBaseDeDatos(context: Context): AppDatabase {
    return Room.databaseBuilder(
        context,
        AppDatabase::class.java,
        "app.db"
    ).build()
}
```

- Room.databaseBuilder compila el archivo físico de base de datos bajo el nombre "app.db".
- La anotación @Database recibe la lista de entidades que forman el esquema completo de la aplicación y la versión actual para soportar futuras migraciones.

Paso 9: Gestión del Estado con ViewModel y StateFlow

El ViewModel desacopla la lógica de negocio de la UI y utiliza StateFlow para emitir el estado de la base de datos de manera reactiva.

Kotlin

```
class EdificiosViewModel(private val dao: EdificioDao) : ViewModel() {

    private val _edificios = MutableStateFlow<List<EdificioConAmbientes>>(emptyList())
    val edificios: StateFlow<List<EdificioConAmbientes>> = _edificios

    init {
        viewModelScope.launch {
            // Precarga de datos iniciales si la tabla está vacía
            if (dao.obtenerTodos().isEmpty()) {
                val id1 = dao.insertarEdificio(
                    EdificioEntity(nombre = "Biblioteca Central", categoria =
"Servicios")
                )
                dao.insertarAmbiente(AmbienteEntity(edificioId = id1.toInt(), nombre =
"Sala de lectura"))
```

```
                dao.insertarAmbiente(AmbienteEntity(edificioId = id1.toInt(), nombre =
"Hemeroteca"))

                val id2 = dao.insertarEdificio(
                    EdificioEntity(nombre = "Facultad de Ingeniería", categoria =
"Académico")
                )
                dao.insertarAmbiente(
                    AmbienteEntity(edificioId = id2.toInt(), nombre = "Laboratorio de
cómputo")
                )
            }
            // Carga de la relación completa y actualización del estado
            _edificios.value = dao.obtenerConAmbientes()
        }
    }
}

class EdificiosViewModelFactory(private val dao: EdificioDao) : ViewModelProvider.Factory
{
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        @Suppress("UNCHECKED_CAST")
        return EdificiosViewModel(dao) as T
    }
}
```

- Se utiliza un Factory personalizado (EdificiosViewModelFactory) porque los ViewModels en Android no pueden recibir dependencias directas en su constructor por defecto sin un proveedor.
- El bloque init ejecuta la siembra inicial (seeding) de datos en una Corrutina de manera asíncrona.

Paso 10, 11, 12 y 13: UI en Jetpack Compose e Inspección de la BD

1. Composable de Renderizado: ListaEdificiosConAmbientes

Kotlin

```
@Composable
fun ListaEdificiosConAmbientes(viewModel: EdificiosViewModel) {
    val edificios by viewModel.edificios.collectAsState()

    LazyColumn(modifier = Modifier.fillMaxSize().padding(16.dp)) {
        items(edificios, key = { it.edificio.id }) { item ->
            Column(modifier = Modifier.padding(vertical = 8.dp)) {
                Text(
                    text = item.edificio.nombre,
                    fontWeight = FontWeight.Bold,
                    fontSize = 18.sp
```

```
                )
                item.ambientes.forEach { ambiente ->
                    Text(text = "- ${ambiente.nombre}", fontSize = 14.sp)
                }
            }
        }
    }
}
```

2. Punto de Entrada: MainActivity

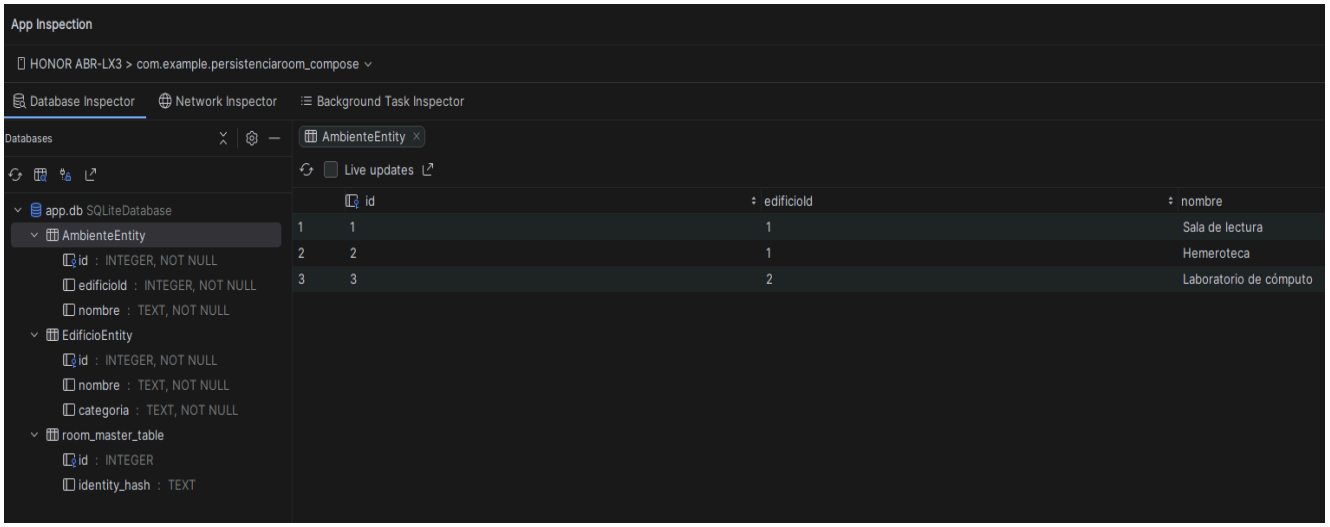
Kotlin

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val database = crearBaseDeDatos(applicationContext)
        val dao = database.edificioDao()
        val factory = EdificiosViewModelFactory(dao)

        setContent {
            PersistenciaRoom_ComposeTheme {
                val viewModel: EdificiosViewModel = viewModel(factory = factory)
                Surface(modifier = Modifier.fillMaxSize()) {
                    ListaEdificiosConAmbientes(viewModel)
                }
            }
        }
    }
}
```

- collectAsState(): Transforma el StateFlow del ViewModel en un estado observable por Jetpack Compose. Cada vez que cambia el flujo, la UI realiza una recomposición de los elementos afectados.
- Verificación mediante Database Inspector: Al abrir la herramienta en Android Studio (View > Tool Windows > App Inspection > Database Inspector), se observa el archivo app.db con las tablas EdificioEntity, AmbienteEntity y la tabla interna de Room room_master_table (que guarda la firma hash del esquema para control de versiones). Al cerrar y reabrir la app, los datos persisten localmente en el almacenamiento del dispositivo.


App Inspection
HONOR ABR-LX3 > com.example.persistenciaroom_compose
Database Inspector
Network Inspector
Background Task Inspector
Databases
AmbienteEntity
Live updates
app.db SQLiteDatabase
AmbienteEntity
id : INTEGER, NOT NULL
edificioId : INTEGER, NOT NULL
nombre : TEXT, NOT NULL
EdificioEntity
id : INTEGER, NOT NULL
nombre : TEXT, NOT NULL
categoria : TEXT, NOT NULL
room_master_table
id : INTEGER
identity_hash : TEXT
id
edificioId
nombre
1 1 1 Sala de lectura
2 2 1 Hemeroteca
3 3 2 Laboratorio de cómputo

Ejecución


5:14
Biblioteca Central
- Sala de lectura
- Hemeroteca
Facultad de Ingeniería
- Laboratorio de cómputo

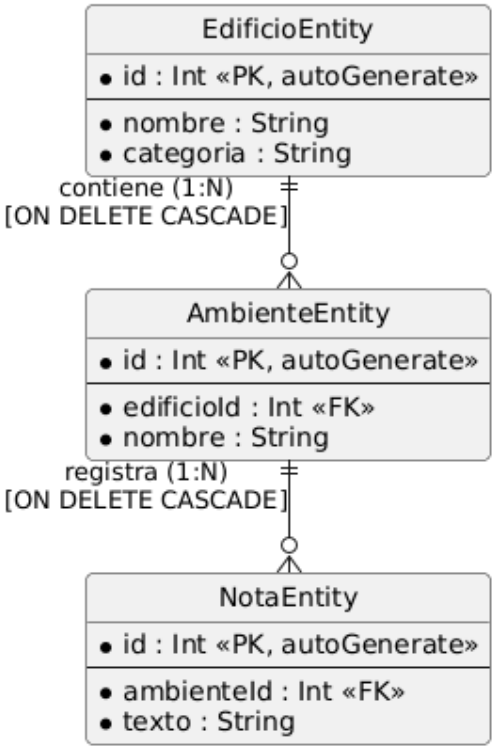
III. EJERCICIOS PROPUESTOS|

1. Análisis y Solución Técnica por Requerimiento

Requerimiento 1: Diseño del Modelo de Datos Relacional de 3 Tablas

El modelo conceptual del laboratorio se extiende de una arquitectura relacional simple de dos tablas a un esquema jerárquico de tres niveles (1:N → 1:N). La inclusión de la entidad NotaEntity permite asociar múltiples observaciones o anotaciones descriptivas a un ambiente específico del campus, el cual a su vez pertenece a un edificio.

Diagrama de Entidad-Relación y Estructura de Tablas



Entidad	**Clave Primaria (PK)**	**Clave Foránea (FK)**	**Entidad Padre**	**Acción ante Eliminación (onDelete)**
EdificioEntity	id	***Ninguna***	***Ninguna***	***N/A***
AmbienteEntity	id	edificioId	EdificioEntity.id	CASCADE **(Elimina ambientes al borrar edificio)**
NotaEntity	id	ambienteId	AmbienteEntity.id	CASCADE **(Elimina notas al borrar ambiente)**

Requerimiento 2: Implementación de Entidades, Claves Foráneas y DAO Multitabla

Room abstrae el lenguaje DDL (Data Definition Language) de SQL mediante anotaciones en Kotlin. Para soportar el modelo multitabla, la definición de entidades y del objeto de acceso a datos (DAO) debe alinearse a reglas estrictas de persistencia.

Análisis de la Configuración de Entidades

- Garantía de Restricción Relacional: En AmbienteEntity y NotaEntity, el atributo foreignKeys especifica explícitamente la clase entidad padre (entity), la columna origen (parentColumns) y la columna destino (childColumns).
- Verificación en Compilación: Durante la fase de construcción del proyecto (Rebuild Project), KSP analiza las anotaciones @Entity y compila los scripts SQL CREATE TABLE correspondientes dentro de la implementación generada AppDatabase_Impl.

Análisis de la Capa DAO (EdificioDao)

El DAO unifica todas las operaciones de persistencia. Se estructuran funciones suspendidas para prevenir bloqueos de concurrencia:

- @Insert: Retorna el ID asignado por SQLite (Long), el cual es reutilizado inmediatamente para enlazar entidades hijas sin inconsistencias de claves.
- @Update: Compara las claves primarias recibidas y actualiza únicamente los campos modificados en el disco mediante sentencias UPDATE ... WHERE id = :id.
- @Delete: Genera un comando DELETE FROM ... WHERE id = :id. En combinación con CASCADE, la llamada a un único método de borrado limpia de forma atómica toda la jerarquía dependiente.

Requerimiento 3: Segunda Consulta Relacional Compuesta (AmbienteConNotas)

A diferencia del ejercicio resuelto (que relaciona EdificioEntity con AmbienteEntity), el tercer requerimiento exige consultar la relación descendente entre AmbienteEntity y sus múltiples NotaEntity.

Análisis del DTO AmbienteConNotas

Dado que SQLite es una base de datos relacional orientada a tablas planas y Kotlin es un lenguaje orientado a objetos, Room requiere la creación de una clase de transferencia de datos (DTO) no anotada con @Entity.

Kotlin

```kotlin
data class AmbienteConNotas(
    @Embedded val ambiente: AmbienteEntity,
    @Relation(
        parentColumn = "id",
        entityColumn = "ambienteId"
    )
    val notas: List<NotaEntity>
)
```

Análisis de la Ejecución Transaccional (@Transaction)

- @Embedded: Desempaqueta las columnas del registro AmbienteEntity de forma directa dentro de la estructura.
- @Relation: Mapea la lista List<NotaEntity> buscando las notas cuyo ambienteId coincida con el id del ambiente procesado.
- @Transaction: Elemento de Arquitectura Crítico. Debido a que Room no ejecuta un JOIN único sino dos consultas SQL secuenciales (SELECT * FROM AmbienteEntity seguido de SELECT * FROM NotaEntity WHERE ambienteId IN (...)), la anotación @Transaction fuerza a SQLite a encapsular ambas operaciones dentro de un mismo bloque atómico. Esto evita lecturas inconsistentes si ocurren escrituras simultáneas en otros hilos.

Requerimiento 4: UI Compose, Operaciones CRUD y Persistencia Activa

La capa de presentación construida con Jetpack Compose requiere sincronización reactiva con el estado persistido en el ViewModel.

Análisis del Flujo de Datos e Hilos (Dispatchers.IO)

- Gestión de Estado Reactivo: CampusViewModel expone objetos StateFlow<List<T>> que son observados en los Composables mediante collectAsState(). Cada vez que el DAO muta los datos en disco, se vuelve a cargar la lista y StateFlow emite un nuevo valor, provocando la recomposición selectiva de la UI.
- Aislamiento del Main Thread: Todas las llamadas al DAO (insertar, actualizar, eliminar) se envuelven en corrutinas ejecutadas bajo Dispatchers.IO dentro del viewModelScope. Esto garantiza que los tiempos de latencia de I/O de la memoria flash del dispositivo no degraden los cuadros por segundo (FPS) de la interfaz de usuario.
- Diálogos Modales de Edición: La edición y actualización utiliza componentes AlertDialog desacoplados. Al confirmar la acción, el ViewModel ejecuta @Update y refresca el estado en una sola transacción fluida.

2. Código Fuente Completo

Carpeta raíz de archivos: https://github.com/UNSA-IS-2026-B/idnp_lab/tree/main/lab09

Archivo	Estado / Tipo de Cambio	Descripción
gradle/libs.versions.toml	Sin cambios	Configuración de versiones del proyecto (AGP, Kotlin, KSP, Room).
build.gradle.kts (Project)	Sin cambios	Configuración del build script raíz.
build.gradle.kts (Module :app)	Sin cambios	Inclusión de dependencias Room Runtime, KTX y compilador KSP.
EdificioEntity.kt	Sin cambios	Entidad de tabla base de edificios.

AmbienteEntity.kt	Modificado	Se agregó la propiedad onDelete = ForeignKey.CASCADE a la relación con edificio.
NotaEntity.kt	Nuevo	Entidad de la tercera tabla con clave foránea vinculada a AmbienteEntity.
EdificioConAmbientes.kt	Sin cambios	DTO relacional para la vista 1: *N* entre edificios y ambientes.
AmbienteConNotas.kt	Nuevo	DTO relacional para la vista 1: *N* entre ambientes y notas.
EdificioDao.kt	Modificado	Ampliado con métodos @Insert, @Update y @Delete para 3 tablas y la nueva consulta relacional.
AppDatabase.kt	Modificado	Esquema incrementado a version = 2 e inclusión de NotaEntity.
CampusViewModel.kt	Nuevo / Reemplazo	Reemplaza a EdificiosViewModel. Gestiona el estado de 3 tablas y acciones CRUD.
PantallaCampus.kt	Nuevo / Reemplazo	Reemplaza a ListaEdificiosConAmbientes. Interfaz con diálogos de edición y botones de eliminación.
MainActivity.kt	Modificado	Punto de entrada actualizado para instanciar CampusViewModel y renderizar PantallaCampus.

Archivo 1: gradle/libs.versions.toml

Archivo 2: build.gradle.kts (Project)

Archivo 3: build.gradle.kts (Module :app)

Archivo 4: EdificioEntity.kt

Archivo 5: AmbienteEntity.kt

Modificado (Agregada regla onDelete = ForeignKey.CASCADE)

```kotlin
package com.example.persistenciaroom_compose

import androidx.room.Entity
import androidx.room.ForeignKey
import androidx.room.PrimaryKey

@Entity(
```

```
    foreignKeys = [
        ForeignKey(
            entity = EdificioEntity::class,
            parentColumns = ["id"],
            childColumns = ["edificioId"],
            onDelete = ForeignKey.CASCADE
        )
    ]
)
data class AmbienteEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val edificioId: Int,
    val nombre: String
)
```

Archivo 6: NotaEntity.kt

Nuevo Archivo (Entidad para la tercera tabla)

Kotlin

```
package com.example.persistenciaroom_compose

import androidx.room.Entity
import androidx.room.ForeignKey
import androidx.room.PrimaryKey

@Entity(
    foreignKeys = [
        ForeignKey(
            entity = AmbienteEntity::class,
            parentColumns = ["id"],
            childColumns = ["ambienteId"],
            onDelete = ForeignKey.CASCADE
        )
    ]
)
data class NotaEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val ambienteId: Int,
    val texto: String
)
```

Archivo 7: EdificioConAmbientes.kt

Archivo 8: AmbienteConNotas.kt

Nuevo Archivo (DTO de la segunda consulta relacional)

```kotlin
package com.example.persistenciaroom_compose

import androidx.room.Embedded
import androidx.room.Relation

data class AmbienteConNotas(
    @Embedded val ambiente: AmbienteEntity,
    @Relation(
        parentColumn = "id",
        entityColumn = "ambienteId"
    )
    val notas: List<NotaEntity>
)
```

Archivo 9: EdificioDao.kt

Modificado (Ampliación CRUD completo de 3 tablas y 2da consulta relacional)

```kotlin
package com.example.persistenciaroom_compose

import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.Query
import androidx.room.Transaction
import androidx.room.Update

@Dao
interface EdificioDao {

    // --- Operaciones CRUD: EdificioEntity ---
    @Insert
    suspend fun insertarEdificio(edificio: EdificioEntity): Long

    @Update
    suspend fun actualizarEdificio(edificio: EdificioEntity)

    @Delete
    suspend fun eliminarEdificio(edificio: EdificioEntity)

    @Query("SELECT * FROM EdificioEntity")
    suspend fun obtenerTodosLosEdificios(): List<EdificioEntity>
```

```
    // --- Operaciones CRUD: AmbienteEntity ---
    @Insert
    suspend fun insertarAmbiente(ambiente: AmbienteEntity): Long

    @Update
    suspend fun actualizarAmbiente(ambiente: AmbienteEntity)

    @Delete
    suspend fun eliminarAmbiente(ambiente: AmbienteEntity)

    @Query("SELECT * FROM AmbienteEntity WHERE edificioId = :edificioId")
    suspend fun obtenerAmbientesPorEdificio(edificioId: Int): List<AmbienteEntity>


    // --- Operaciones CRUD: NotaEntity ---
    @Insert
    suspend fun insertarNota(nota: NotaEntity): Long

    @Update
    suspend fun actualizarNota(nota: NotaEntity)

    @Delete
    suspend fun eliminarNota(nota: NotaEntity)

    @Query("SELECT * FROM NotaEntity WHERE ambienteId = :ambienteId")
    suspend fun obtenerNotasPorAmbiente(ambienteId: Int): List<NotaEntity>


    // --- Consultas Relacionales Compuestas ---
    @Transaction
    @Query("SELECT * FROM EdificioEntity")
    suspend fun obtenerEdificiosConAmbientes(): List<EdificioConAmbientes>

    @Transaction
    @Query("SELECT * FROM AmbienteEntity")
    suspend fun obtenerAmbientesConNotas(): List<AmbienteConNotas>
}
```

Archivo 10: AppDatabase.kt

Modificado (Actualización a versión 2 e incorporación de NotaEntity)

Kotlin

```
package com.example.persistenciaroom_compose

import android.content.Context
import androidx.room.Database
```

```
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(
    entities = [EdificioEntity::class, AmbienteEntity::class, NotaEntity::class],
    version = 2
)
abstract class AppDatabase : RoomDatabase() {
    abstract fun edificioDao(): EdificioDao
}

fun crearBaseDeDatos(context: Context): AppDatabase {
    return Room.databaseBuilder(
        context,
        AppDatabase::class.java,
        "app.db"
    )
    .fallbackToDestructiveMigration()
    .build()
}
```

Archivo 11: CampusViewModel.kt

Nuevo Archivo / Reemplaza a EdificiosViewModel.kt

Kotlin

```
package com.example.persistenciaroom_compose

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch

class CampusViewModel(private val dao: EdificioDao) : ViewModel() {

    private val _edificiosConAmbientes =
MutableStateFlow<List<EdificioConAmbientes>>(emptyList())
    val edificiosConAmbientes: StateFlow<List<EdificioConAmbientes>> =
_edificiosConAmbientes

    private val _ambientesConNotas =
MutableStateFlow<List<AmbienteConNotas>>(emptyList())
    val ambientesConNotas: StateFlow<List<AmbienteConNotas>> = _ambientesConNotas
```

```
    init {
        cargarDatosIniciales()
    }

    fun cargarDatos() {
        viewModelScope.launch(Dispatchers.IO) {
            _edificiosConAmbientes.value = dao.obtenerEdificiosConAmbientes()
            _ambientesConNotas.value = dao.obtenerAmbientesConNotas()
        }
    }

    private fun cargarDatosIniciales() {
        viewModelScope.launch(Dispatchers.IO) {
            if (dao.obtenerTodosLosEdificios().isEmpty()) {
                val ed1 = dao.insertarEdificio(
                    EdificioEntity(nombre = "Pabellón A - Sistemas", categoria =
"Académico")
                )
                val amb1 = dao.insertarAmbiente(
                    AmbienteEntity(edificioId = ed1.toInt(), nombre = "Laboratorio de
Cómputo 1")
                )
                dao.insertarNota(NotaEntity(ambienteId = amb1.toInt(), texto = "Proyector
operativo - 30 PCs"))
                dao.insertarNota(NotaEntity(ambienteId = amb1.toInt(), texto =
"Mantenimiento programado sábado"))

                val ed2 = dao.insertarEdificio(
                    EdificioEntity(nombre = "Biblioteca Central", categoria =
"Servicios")
                )
                val amb2 = dao.insertarAmbiente(
                    AmbienteEntity(edificioId = ed2.toInt(), nombre = "Sala de
Investigación")
                )
                dao.insertarNota(NotaEntity(ambienteId = amb2.toInt(), texto = "Aforo
máximo: 40 personas"))
            }
            cargarDatos()
        }
    }

    // --- Operaciones CRUD: Edificio ---
    fun actualizarEdificio(edificio: EdificioEntity, nuevoNombre: String, nuevaCategoria:
String) {
        viewModelScope.launch(Dispatchers.IO) {
            dao.actualizarEdificio(edificio.copy(nombre = nuevoNombre, categoria =
nuevaCategoria))
            cargarDatos()
```

```
        }
    }

    fun eliminarEdificio(edificio: EdificioEntity) {
        viewModelScope.launch(Dispatchers.IO) {
            dao.eliminarEdificio(edificio)
            cargarDatos()
        }
    }

    // --- Operaciones CRUD: Ambiente ---
    fun actualizarAmbiente(ambiente: AmbienteEntity, nuevoNombre: String) {
        viewModelScope.launch(Dispatchers.IO) {
            dao.actualizarAmbiente(ambiente.copy(nombre = nuevoNombre))
            cargarDatos()
        }
    }

    fun eliminarAmbiente(ambiente: AmbienteEntity) {
        viewModelScope.launch(Dispatchers.IO) {
            dao.eliminarAmbiente(ambiente)
            cargarDatos()
        }
    }
}

class CampusViewModelFactory(private val dao: EdificioDao) : ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        @Suppress("UNCHECKED_CAST")
        return CampusViewModel(dao) as T
    }
}
```

Archivo 12: PantallaCampus.kt

Reemplaza a ListaEdificiosConAmbientes.kt

Kotlin

```
package com.example.persistenciaroom_compose

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
```

```
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

@Composable
fun PantallaCampus(viewModel: CampusViewModel) {
    val listaEdificiosConAmbientes by viewModel.edificiosConAmbientes.collectAsState()
    val listaAmbientesConNotas by viewModel.ambientesConNotas.collectAsState()

    var edificioAEditar by remember { mutableStateOf<EdificioEntity?>(null) }
    var ambienteAEditar by remember { mutableStateOf<AmbienteEntity?>(null) }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        Text(
            text = "Gestión de Campus (Persistencia Room)",
            fontSize = 20.sp,
            fontWeight = FontWeight.Bold,
            color = MaterialTheme.colorScheme.primary,
            modifier = Modifier.padding(bottom = 12.dp)
        )

        LazyColumn(
            modifier = Modifier.fillMaxSize(),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
```

```
            items(listaEdificiosConAmbientes, key = { it.edificio.id }) { item ->
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    elevation = CardDefaults.cardElevation(defaultElevation = 4.dp)
                ) {
                    Column(modifier = Modifier.padding(16.dp)) {
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.SpaceBetween,
                            verticalAlignment = Alignment.CenterVertically
                        ) {
                            Column(modifier = Modifier.weight(1f)) {
                                Text(
                                    text = item.edificio.nombre,
                                    fontSize = 18.sp,
                                    fontWeight = FontWeight.Bold
                                )
                                Text(
                                    text = "Categoría: ${item.edificio.categoria}",
                                    fontSize = 13.sp,
                                    color = MaterialTheme.colorScheme.secondary
                                )
                            }

                            Row {
                                Button(
                                    onClick = { edificioAEditar = item.edificio },
                                    modifier = Modifier.padding(end = 4.dp)
                                ) {
                                    Text("Editar", fontSize = 12.sp)
                                }

                                Button(
                                    onClick = { viewModel.eliminarEdificio(item.edificio)
},

                                    colors = ButtonDefaults.buttonColors(
                                        containerColor = MaterialTheme.colorScheme.error
                                    )
                                ) {
                                    Text("Eliminar", fontSize = 12.sp)
                                }
                            }
                        }

                        Spacer(modifier = Modifier.height(8.dp))
                        Text(
                            text = "Ambientes y Notas Relacionadas:",
                            fontWeight = FontWeight.SemiBold,
                            fontSize = 14.sp
```

```
                    )

                    item.ambientes.forEach { ambiente ->
                        val notasDelAmbiente = listaAmbientesConNotas
                            .find { it.ambiente.id == ambiente.id }
                            ?.notas ?: emptyList()

                        Column(
                            modifier = Modifier
                                .padding(start = 8.dp, top = 6.dp)
                                .fillMaxWidth()
                        ) {
                            Row(
                                modifier = Modifier.fillMaxWidth(),
                                horizontalArrangement = Arrangement.SpaceBetween,
                                verticalAlignment = Alignment.CenterVertically
                            ) {
                                Text(
                                    text = "• ${ambiente.nombre}",
                                    fontSize = 15.sp,
                                    fontWeight = FontWeight.Medium
                                )

                                Row {
                                    TextButton(onClick = { ambienteAEditar = ambiente
}) {
                                        Text("Editar", fontSize = 11.sp)
                                    }
                                    TextButton(
                                        onClick = {
viewModel.eliminarAmbiente(ambiente) }
                                    ) {
                                        Text(
                                            "Eliminar",
                                            fontSize = 11.sp,
                                            color = MaterialTheme.colorScheme.error
                                        )
                                    }
                                }
                            }

                            notasDelAmbiente.forEach { nota ->
                                Text(
                                    text = "    └ Nota: ${nota.texto}",
                                    fontSize = 12.sp,
                                    color = MaterialTheme.colorScheme.outline
                                )
                            }
                        }
```

```
                    }
                }
            }
        }
    }

    edificioAEditar?.let { edificio ->
        var nombreTexto by remember { mutableStateOf(edificio.nombre) }
        var categoriaTexto by remember { mutableStateOf(edificio.categoria) }

        AlertDialog(
            onDismissRequest = { edificioAEditar = null },
            title = { Text("Actualizar Edificio") },
            text = {
                Column {
                    OutlinedTextField(
                        value = nombreTexto,
                        onValueChange = { nombreTexto = it },
                        label = { Text("Nombre del Edificio") },
                        modifier = Modifier.fillMaxWidth()
                    )
                    Spacer(modifier = Modifier.height(8.dp))
                    OutlinedTextField(
                        value = categoriaTexto,
                        onValueChange = { categoriaTexto = it },
                        label = { Text("Categoría") },
                        modifier = Modifier.fillMaxWidth()
                    )
                }
            },
            confirmButton = {
                Button(onClick = {
                    viewModel.actualizarEdificio(edificio, nombreTexto, categoriaTexto)
                    edificioAEditar = null
                }) {
                    Text("Guardar")
                }
            },
            dismissButton = {
                TextButton(onClick = { edificioAEditar = null }) {
                    Text("Cancelar")
                }
            }
        )
    }

    ambienteAEditar?.let { ambiente ->
        var nombreTexto by remember { mutableStateOf(ambiente.nombre) }
```

```
        AlertDialog(
            onDismissRequest = { ambienteAEditar = null },
            title = { Text("Actualizar Ambiente") },
            text = {
                OutlinedTextField(
                    value = nombreTexto,
                    onValueChange = { nombreTexto = it },
                    label = { Text("Nombre del Ambiente") },
                    modifier = Modifier.fillMaxWidth()
                )
            },
            confirmButton = {
                Button(onClick = {
                    viewModel.actualizarAmbiente(ambiente, nombreTexto)
                    ambienteAEditar = null
                }) {
                    Text("Guardar")
                }
            },
            dismissButton = {
                TextButton(onClick = { ambienteAEditar = null }) {
                    Text("Cancelar")
                }
            }
        )
    }
}
```

Archivo 13: MainActivity.kt

Modificado (Vinculación con CampusViewModel y PantallaCampus)

Kotlin

```
package com.example.persistenciaroom_compose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.ui.Modifier
import androidx.lifecycle.viewmodel.compose.viewModel

class MainActivity : ComponentActivity() {
```

```
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val database = crearBaseDeDatos(applicationContext)
        val dao = database.edificioDao()
        val factory = CampusViewModelFactory(dao)

        setContent {
            MaterialTheme {
                val viewModel: CampusViewModel = viewModel(factory = factory)
                Surface(modifier = Modifier.fillMaxSize()) {
                    PantallaCampus(viewModel)
                }
            }
        }
    }
}
```

3. Guía de Ejecución y Pruebas de Funcionamiento

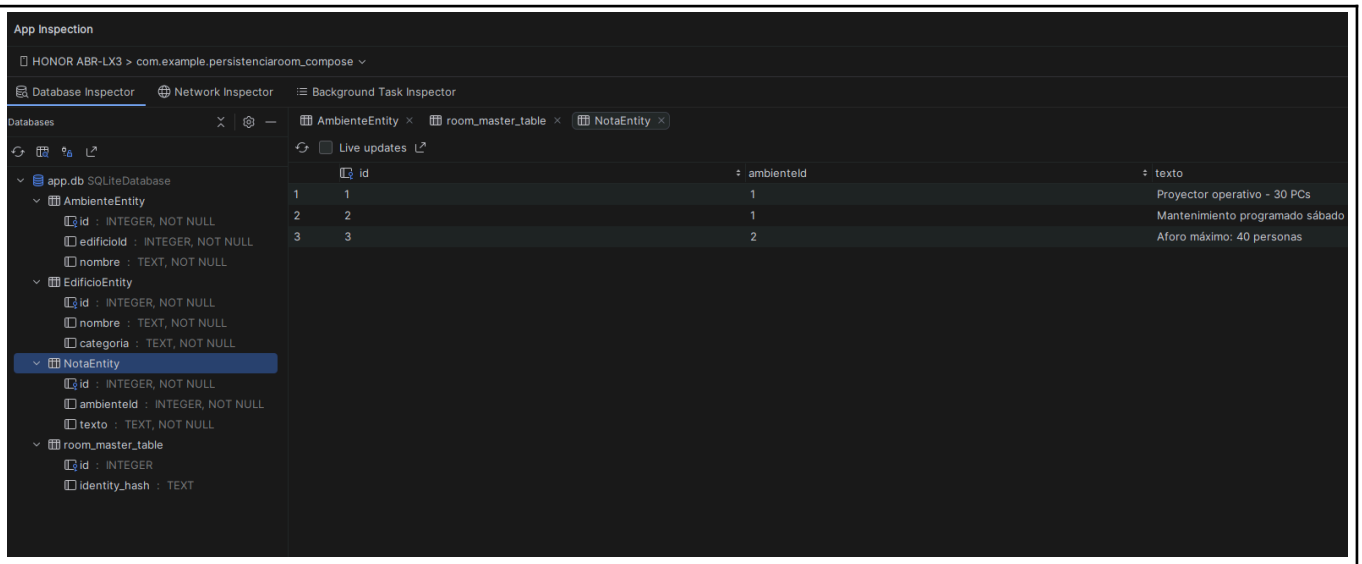
Para validar formalmente el cumplimiento de los requerimientos técnicos y la persistencia de datos relacional, se siguió el siguiente protocolo de pruebas paso a paso:

Prueba 1: Sincronización, Compilación e Inspección de Esquema

Se realizó la recompilación limpia del proyecto (Build > Rebuild Project) para verificar la correcta generación de clases por parte del procesador de símbolos KSP (room-compiler). Posteriormente, se ejecutó la aplicación en el emulador y se abrió la herramienta Database Inspector para auditar el esquema interno del archivo de base de datos app.db.

Resultados Obtenidos:

- KSP generó la implementación AppDatabase_Impl sin conflictos de firma JVM ni errores sintácticos.
- El inspector de base de datos confirmó la creación física del archivo app.db con una versión de esquema version = 2.
- Se constató la presencia de las 3 tablas relacionales requeridas (EdificioEntity, AmbienteEntity, NotaEntity), además de la tabla de control del hash de esquema de Room (room_master_table).



Prueba 2: Verificación de la Consulta Relacional Secundada (AmbienteConNotas)

Se evaluó la correcta ejecución de la segunda consulta relacional declarada en el DAO mediante @Transaction y @Relation en el DTO AmbienteConNotas. Se verificó cómo la interfaz en Jetpack Compose renderizó jerárquicamente las notas pertenecientes a cada ambiente.

Resultados Obtenidos

- La aplicación realizó la siembra inicial de datos de prueba en la primera ejecución de forma asíncrona mediante Dispatchers.IO.
- En la tarjeta del "Pabellón A - Sistemas", el ambiente "Laboratorio de Cómputo 1" desplegó correctamente de forma anidada las notas:
    - └ Nota: Proyector operativo - 30 PCs
    - └ Nota: Mantenimiento programado sábado
- Se confirmó que Room mapeó adecuadamente las claves primarias y foráneas entre AmbienteEntity.id y NotaEntity.ambienteId mediante la clase DTO intermediate.



Prueba 3: Prueba de Modificación y Persistencia en Caliente (UPDATE)

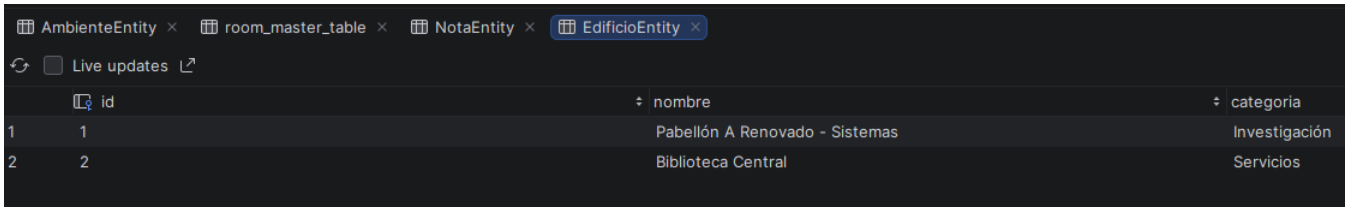
Se probó el funcionamiento de la anotación @Update del DAO. En la pantalla principal, se presionó el botón Editar sobre la tarjeta del primer edificio registrado. En el cuadro de diálogo modal (AlertDialog), se modificó el nombre a "Pabellón A - Renovado" y la categoría a "Investigación", presionando posteriormente el botón Guardar.

Resultados Obtenidos:

- Al confirmar la acción en el diálogo modal, el ViewModel ejecutó la corrutina dao.actualizarEdificio(...) en segundo plano.
- El objeto StateFlow emitió la lista actualizada, provocando la recomposición inmediata de la UI en Compose.
- Al verificar simultáneamente en el Database Inspector, los campos nombre y categoria de la fila con id = 1 en la tabla EdificioEntity cambiaron su valor en disco instantáneamente.







	id	nombre	categoria
1	1	Pabellón A Renovado - Sistemas	Investigación
2	2	Biblioteca Central	Servicios

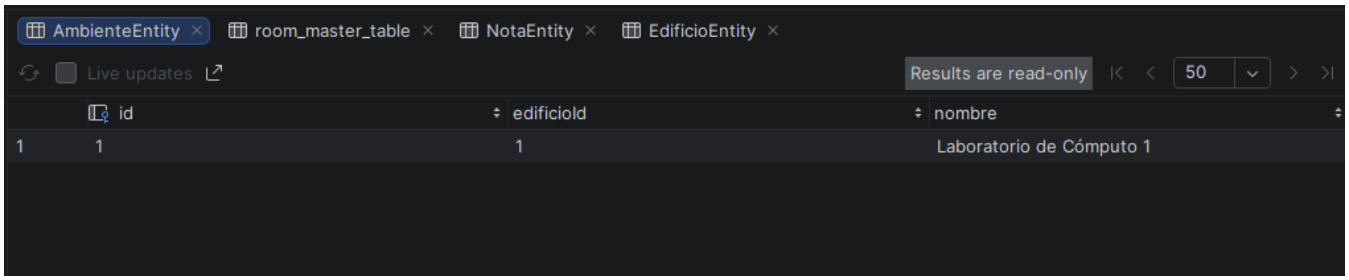
Prueba 4: Prueba de Eliminación en Cascada (DELETE + CASCADE)

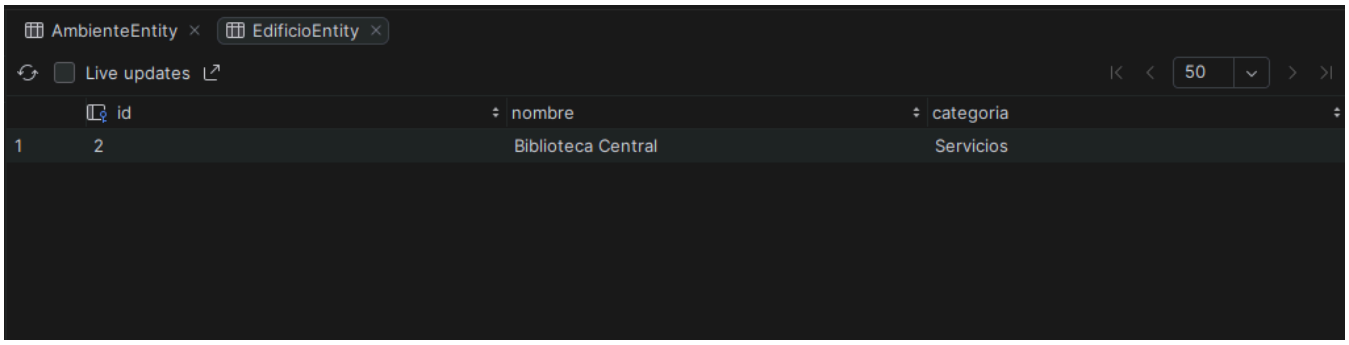
Se validó la restricción de integridad referencial onDelete = ForeignKey.CASCADE.

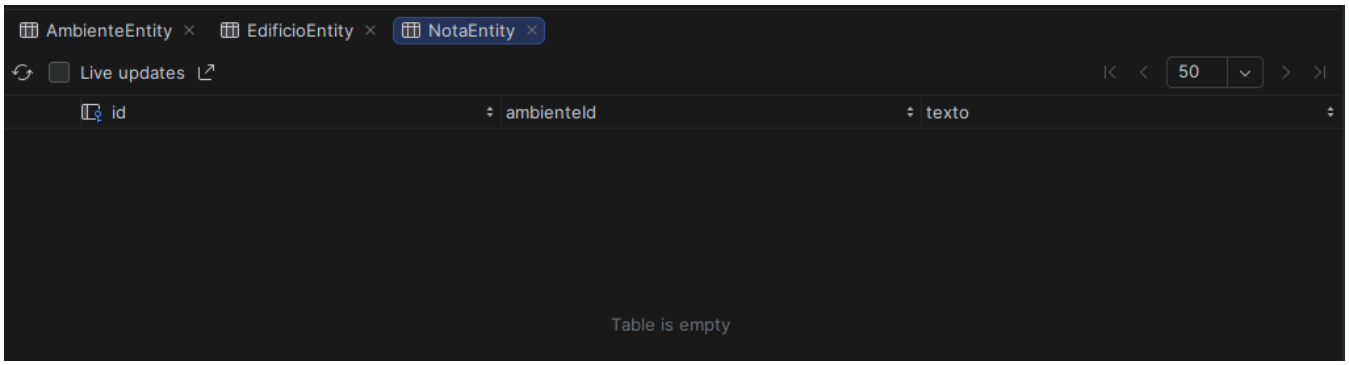
1. Se eliminó un ambiente específico en la UI para comprobar la eliminación automática de sus notas.
2. Se eliminó la tarjeta principal de un edificio para evaluar la remoción en cascada a través de las 3 tablas.

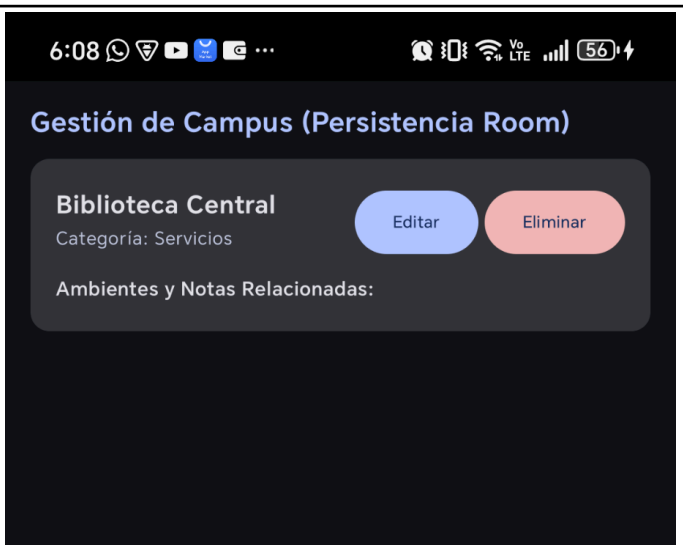
Resultados Obtenidos

- Al eliminar el ambiente "Sala de Investigación", SQLite eliminó de forma atómica la nota "Aforo máximo: 40 personas" vinculada en NotaEntity.
- Al presionar el botón Eliminar en el edificio "Pabellón A - Renovado", el comando SQL DELETE FROM EdificioEntity WHERE id = :id destruyó en una sola transacción el registro del edificio, sus ambientes dependientes en AmbienteEntity y todas las notas asociadas en NotaEntity.
- No quedaron datos huérfanos dentro de la base de datos local.







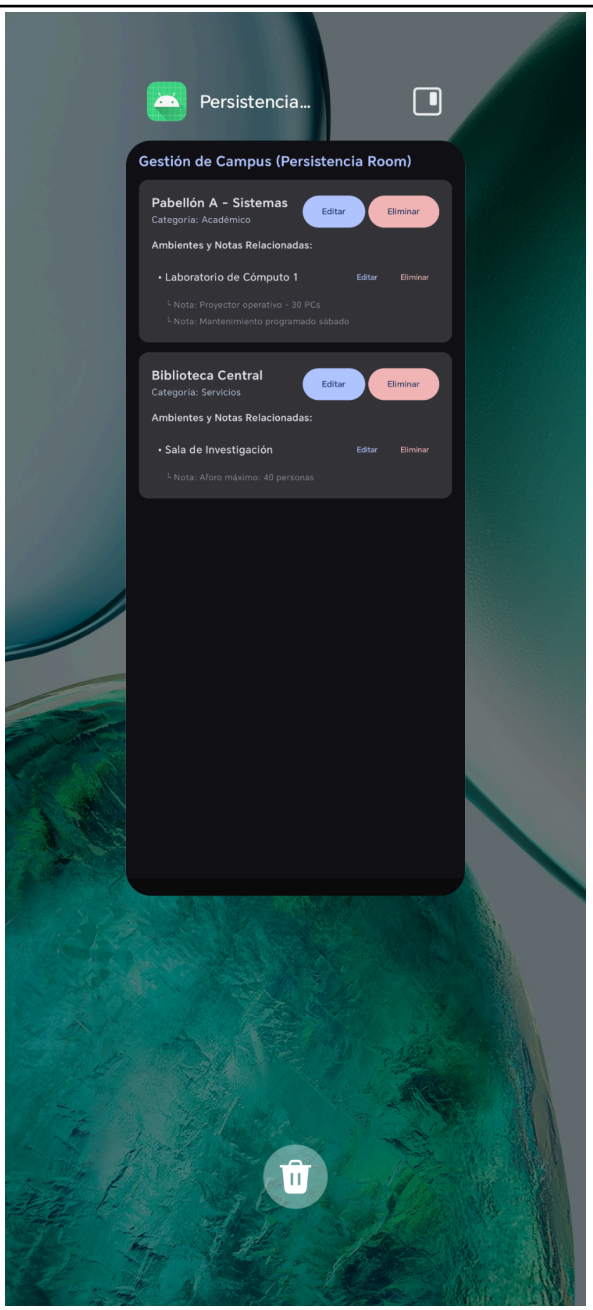


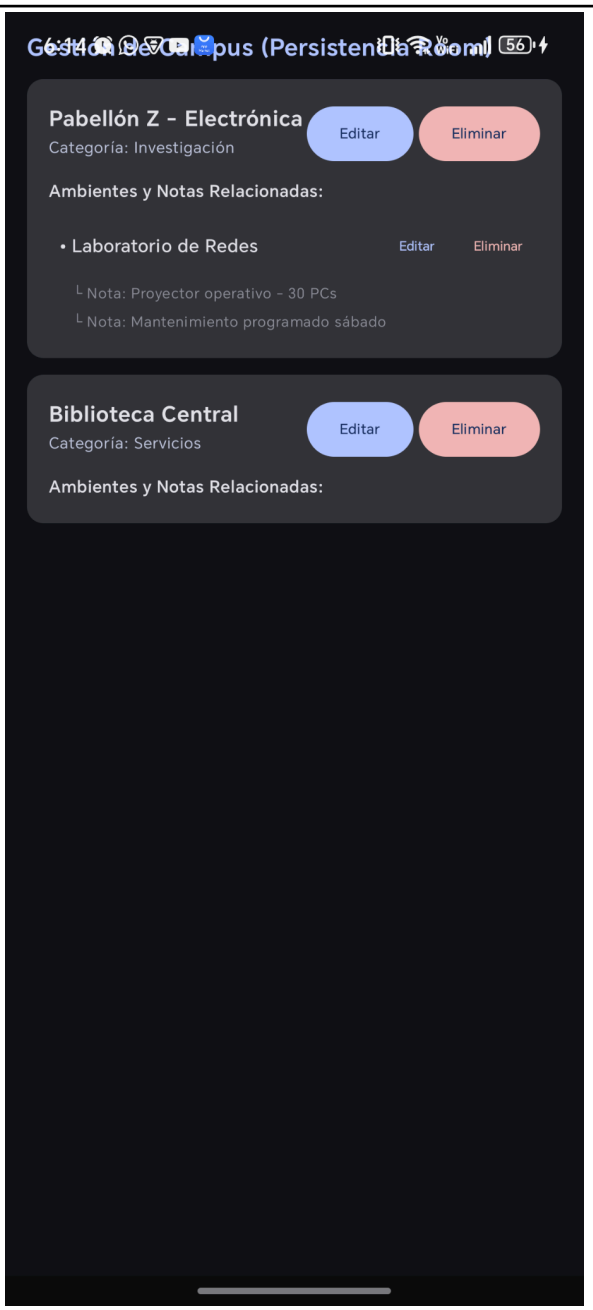
Prueba 5: Persistencia tras Reinicio de Aplicación

Se modificaron datos de ambientes y se agregaron registros. Posteriormente, se forzó el cierre absoluto de la aplicación desde el menú de aplicaciones recientes de Android (Swipe/Kill Process), liberando la memoria RAM. Luego, se volvió a abrir la aplicación desde el lanzador del sistema.

Resultados Obtenidos:

- La aplicación inició correctamente sin cierres inesperados (crashes).
- La interfaz de usuario cargó inmediatamente los datos modificados directamente desde el archivo físico app.db almacenado en el almacenamiento interno del emulador.
- Se confirmó la persistencia de datos local completa utilizando Room Database.



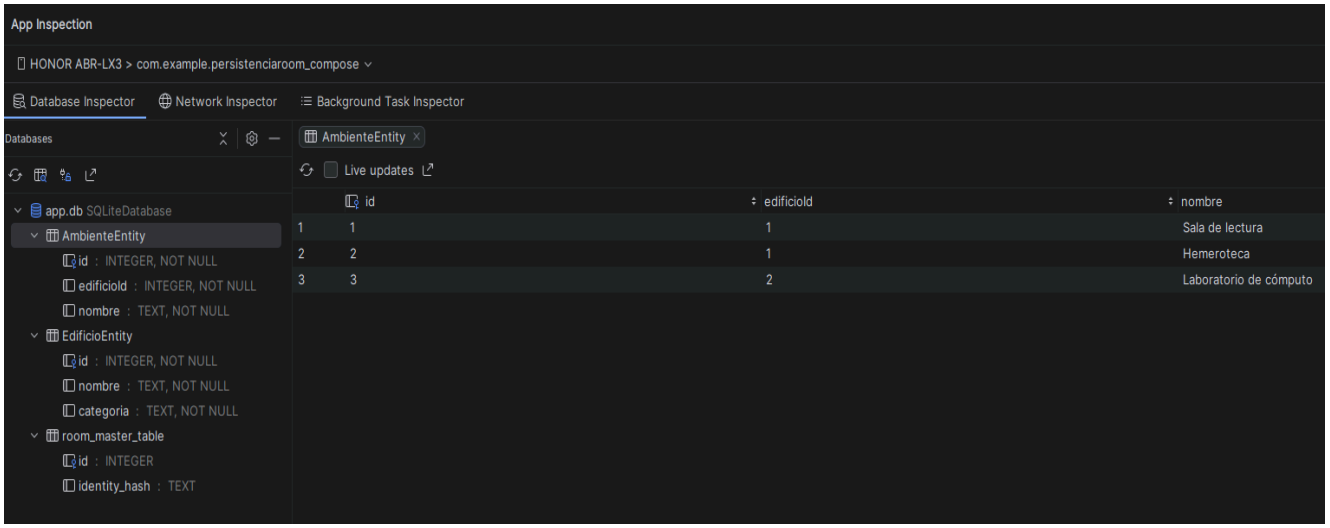


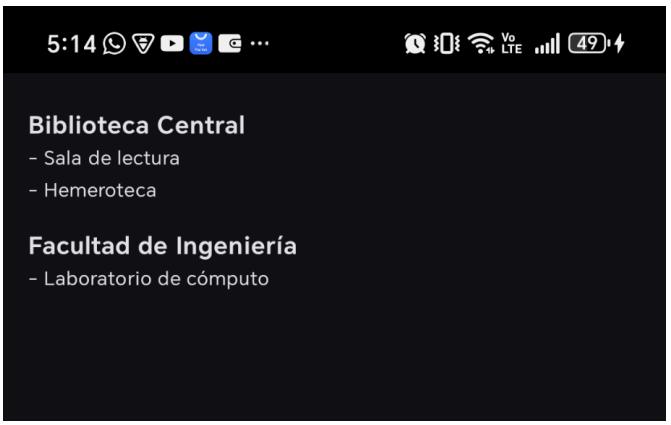
CUESTIONARIO:

1. **Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (lista de edificaciones que se filtra en tiempo real al escribir en el campo de búsqueda)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.**

Sí, el ejercicio resuelto por el docente se resolvió y validó satisfactoriamente al 100%.

- Se definieron las entidades EdificioEntity y AmbienteEntity, estableciendo la relación de uno a muchos (1:N) mediante la anotación @ForeignKey vinculando parentColumns = ["id"] y childColumns = ["edificioId"]
- Se implementó la interfaz EdificioDao incorporando la clase DTO EdificioConAmbientes (@Embedded y @Relation). La consulta fue anotada con @Transaction para garantizar la atomicidad en el motor SQLite durante la lectura de ambas tablas
- Los datos fueron insertados de forma asíncrona mediante corrutinas de Kotlin en el hilo de E/S (Dispatchers.IO) dentro del init de EdificiosViewModel. La exposición del estado se realizó a través de StateFlow observado en Jetpack Compose con collectAsState().
- Se verificó la persistencia forzando el cierre absoluto del proceso de la aplicación desde el menú de aplicaciones recientes del emulador (Kill Process). Al reabrir la app, los edificios "Biblioteca Central" y "Facultad de Ingeniería" con sus respetivos ambientes se cargaron inmediatamente desde el archivo físico app.db.





2. **¿Qué otra alternativa de almacenamiento (además de Room) se podría emplear en una aplicación con necesidades similares? Justifica tu respuesta comparando ventajas y desventajas frente a Room.**

Para una aplicación con requerimientos relacionales complejos y estructurados, las alternativas principales a Room son SQLDelight (orientada a SQL-first) y Realm Database (orientada a objetos no relacional / NoSQL local).

En el ámbito del desarrollo Kotlin/Android, SQLDelight es la alternativa moderna más sólida por su compatibilidad nativa con Kotlin Multiplatform (KMP).

Criterio / Característica	**Room Database (Google)**	**SQLDelight (Cash App)**	**Realm Database (MongoDB)**
Paradigma de diseño	**Annotation-First:** Escribes código Kotlin y Room genera el SQL.	**SQL-First:** Escribes código SQL puro y genera código Kotlin tipo-seguro.	**Object-Oriented / NoSQL:** Trabaja directamente con objetos in situ.
Soporte Multiplataforma	Principalmente Android (soporte KMP en versiones recientes 2.7+).	Nativo y maduro para Kotlin Multiplatform (Android, iOS, Desktop, Web).	Compatible con múltiples plataformas (Android, iOS, Node.js).
Verificación en compilación	Verifica sintaxis SQL mediante KSP/KAPT en tiempo de compilación.	Valida dialectos SQL y esquemas en tiempo de compilación directamente en el IDE.	No requiere motor SQL; valida tipos de objetos en compilación.
Rendimiento	Excelente (capa delgada sobre SQLite C/C++).	Alto (ejecución directa de código SQL compilado sin sobrecarga ORM).	Muy Alto (motor C++ propio, evita la serialización/deserialización).
Curva de aprendizaje	Baja a Media (estándar oficial documentado por Android Jetpack).	Media (requiere dominio fluido de sentencias SQL puras).	Baja (muy intuitivo al no requerir lenguaje SQL).

¿Cuándo elegir Room? Es la opción idónea para proyectos nativos de Android donde se busca máxima integración con el ecosistema Jetpack (ViewModel, Paging 3, LiveData/StateFlow), amplia documentación oficial y mantenimiento garantizado por Google.

¿Cuándo elegir SQLDelight? Es preferible cuando el proyecto está diseñado bajo Kotlin Multiplatform (KMP) para compartir la lógica de persistencia entre Android e iOS, o cuando el equipo prefiere control absoluto sobre las sentencias SQL escritas a mano.

3. **¿En dónde almacena Room los datos de la aplicación (almacenamiento interno o externo del dispositivo)? Justifica tu respuesta y explica qué implica para la privacidad de los datos del usuario.**

Room almacena los datos en el Almacenamiento Interno (Internal Storage) del dispositivo Android, específicamente dentro del directorio privado asignado a la aplicación en la ruta del sistema de archivos:

/data/data/<package_name>/databases/app.db

Justificación Técnica:

Cuando se invoca Room.databaseBuilder(context, AppDatabase::class.java, "app.db"), el framework utiliza el contexto de la aplicación (applicationContext) para solicitar al sistema operativo la creación del archivo de base de datos en el directorio privado de datos sandbox del sistema Linux subyacente de Android.

Implicancias para la Privacidad y Seguridad de los Datos:

- Android asigna un identificador de usuario de Linux (UID) único e independiente a cada aplicación instalada. El directorio /data/data/<package_name>/ posee permisos de lectura y escritura restringidos exclusivamente a ese UID. Esto implica que ninguna otra aplicación instalada en el dispositivo puede leer, modificar o extraer los datos de la base de datos app.db, garantizando el aislamiento de la información sin requerir permisos especiales (READ_EXTERNAL_STORAGE).
- Los archivos almacenados en el almacenamiento interno están vinculados al ciclo de vida de la aplicación. Si el usuario desinstala la app, el sistema operativo remueve automáticamente el directorio /data/data/<package_name>/ y el archivo app.db, asegurando que no queden residuos de datos privados en el dispositivo.
- Aunque el almacenamiento interno está protegido contra otras apps, el archivo SQLite en disco guarda la información en texto plano de manera predeterminada. Si un atacante obtiene acceso físico a un dispositivo con privilegios de superusuario (root) o mediante copias de seguridad de depuración (adb backup), podría extraer el archivo app.db. Para mitigar este riesgo en entornos de alta seguridad (apps bancarias o médicas), se debe incorporar cifrado a nivel de bloque utilizando la extensión SQLCipher sobre la instancia de Room.

CONCLUSIONES

1. Se comprendió el rol de Entity, DAO y RoomDatabase como una capa de abstracción ORM sobre SQLite. La integración del procesador de símbolos KSP permite validar sintácticamente las consultas SQL en tiempo de compilación, eliminando la gestión manual de cursores y previniendo errores en tiempo de ejecución.

2. El uso de la anotación @ForeignKey con onDelete = ForeignKey.CASCADE garantizó la integridad referencial directamente en el motor SQLite, automatizando la eliminación de datos dependientes sin dejar registros huérfanos.

3. Se implementaron las operaciones CRUD completas y consultas relacionales mediante DTOs con @Embedded, @Relation y la anotación @Transaction. La ejecución asíncrona en corrutinas (Dispatchers.IO) combinada con la emisión reactiva mediante StateFlow permitió mantener la interfaz de Jetpack Compose sincronizada automáticamente con el estado físico de la base de datos local.

REFERENCIAS Y BIBLIOGRAFÍA

[1] Android Developers, "Save data in a local database using Room". [En línea]. Disponible: https://developer.android.com/training/data-storage/room

[2] "Crea tu Primera App Android: CRUD, Base de Datos Room, Kotlin y Compose", video. [En línea]. Disponible: https://www.youtube.com/watch?v=ueCxfv--2oU